Diffusion Models: From Adding Noise to Generating Images
Understanding Denoising-Based Generation through DDPM and LDM

[Full name 1] – [Student ID 1] [Full name 2] – [Student ID 2]
[Full name 3] – [Student ID 3]
University of Science (HCMUS), VNU-HCM
CS420 – Artificial Intelligence

Abstract

How can learning to remove noise enable a model to generate new images? This report explains the connection between Gaussian corruption, score estimation, noise-prediction training, and iterative generation. We review two complementary papers: Denoising Diffusion Probabilistic Models (DDPM) establishes a practical pixel-space formulation, while Latent Diffusion Models (LDM) moves denoising into a learned compressed representation. We explain the origin and role of DDPM's simplified loss, distinguish training from sampling, and illustrate forward noising with a complete scalar calculation and an original five-level image sequence. Published DDPM results demonstrate a sample-quality benefit from objective reweighting; a separately attributed LDM inpainting comparison illustrates the computational benefit of latent diffusion. The discussion emphasizes VAE encoder/decoder roles, briefly introduces conditioning, and separates compression savings from sampler acceleration. Mathematical derivations are provided in the appendices. No trained or pretrained image generator was executed for this report.

1. Introduction

Adding noise destroys visible image structure, yet learning to remove it can support a generative model. The apparent paradox is resolved by considering a distribution of images rather than recovering one particular corrupted photograph. Across many training examples and noise levels, a denoiser learns statistical information that can guide a new random state toward plausible images.

Our guiding question is: *How can learning to remove noise enable a model to generate new images?* Our learning objective is to *explain forward noising and reverse denoising, including why latent diffusion reduces computational demands*. These goals organize the report around two complementary papers. DDPM [6] explains how Gaussian transitions and noise prediction define a practical generative process. LDM [10] asks how to retain that process while reducing repeated computation at image resolution.

We first establish the conceptual bridge from distribution modeling to noise prediction, then study DDPM's formulation and worked examples, and finally examine LDM's encoder, decoder, and latent representation. Results distinguish our own forward-noising illustrations from published model measurements. We do not train a denoiser or run a pretrained generator, GPU benchmark, or prompt-adherence experiment.

2. Background and Paper Review

2.1. Generative versus discriminative models

The fundamental distinction between discriminative and generative models lies in what they aim to learn from data. A discriminative model learns to predict a target y given an observation x, typically by estimating $p_\omega(y \mid x)$ or learning a decision boundary between classes. For instance, a cat classifier only needs to identify features that distinguish cats from other objects, without explicitly modeling the distribution of natural images. In contrast, a generative model aims to capture the underlying data distribution, such as $p_\theta(x)$ or the conditional distribution $p_\theta(x \mid y)$. Rather than merely distinguishing categories, it must model the statistical structure and variability of the data, including factors such as shape, texture, lighting, and spatial relationships. This enables the model to generate new samples that resemble the training data, potentially conditioned on labels or text descriptions. Consequently, generative modeling generally requires capturing richer information about the data than is necessary for classification.

2.2. Qualitative Comparison of Generative Models

GANs, autoregressive models, and diffusion models differ in how they learn and sample the data distribution. A *GAN* trains a generator to transform random noise into images while a discriminator learns to distinguish generated images from real ones [3]. Sampling typically takes one generator

pass, but training requires balancing the two competing networks. An *autoregressive model* generates an image one pixel or token at a time, predicting each from the previously generated values [16]. This gives an explicit factorization of the image distribution, although sequential generation can be costly. A *diffusion model* learns to reverse gradual noise corruption and generates an image by repeatedly denoising an initially random state [6]. It refines the whole state across multiple steps, requiring repeated network evaluations during sampling. Table 1 summarizes these learning ideas, sampling procedures, and qualitative trade-offs.

2.3. Historical context: diffusion and score-based modeling

Sohl-Dickstein et al. [12] introduced a tractable probabilistic construction that gradually corrupts data and learns a reverse process. Song and Ermon [14] developed a score-based route: estimate gradients of noisy data distributions at multiple noise levels and generate samples using annealed Langevin dynamics. DDPM connects diffusion modeling to denoising score matching through its noise-prediction parameterization [6]. LDM later changes the representation on which diffusion operates [10].

These contributions provide context rather than a claim that DDPM was derived historically by following a single score-first chain. The DDPM paper starts from a probabilistic model and variational inference. The next section uses the score viewpoint as a conceptual explanation; Section 4 then returns to DDPM's own formulation.

3. Method: From Denoising to Generation

3.1. From a data distribution to its score

Let $p_{\text{data}}(x)$ denote the unknown distribution of normalized image vectors $x \in \mathbb{R}^D$. A generative model should produce samples with a distribution close to p_{data}; it need not evaluate the true data density explicitly. For a smooth positive density p, its *score* is

$$s(x) = \nabla_x \log p(x). \tag{1}$$

The score is a vector pointing toward the steepest local increase in log density, not a scalar quality rating. For the one-dimensional example $p(x) = \mathcal{N}(0, 1)$, $\log p(x) = -x^2/2 + C$ and $s(x) = -x$: positive values are directed left and negative values right.

A score can support sampling rather than only optimization. For example, a discretized Langevin update uses the score and fresh Gaussian noise [14]:

$$v^{(k+1)} = v^{(k)} + \frac{\delta}{2} s(v^{(k)}) + \sqrt{\delta}\, \zeta^{(k)}. \tag{2}$$

Here $v^{(k)}$ is the sampling state after k updates, $\delta > 0$ is a step size, and $\zeta^{(k)} \sim \mathcal{N}(0, \mathbf{I})$ are independent draws. The random term matters: moving only up the density gradient tends to seek modes, whereas generation must represent a distribution. Exact sampling requires suitable limiting and regularity conditions; a finite update sequence is an approximation.

3.2. Why learn at multiple Gaussian noise levels?

Real images can concentrate near a low-dimensional subset of the ambient pixel space, and scores away from observed data can be difficult to estimate. Gaussian perturbation smooths the distribution and supplies tractable training targets [14]. To explain the connection without yet choosing a DDPM schedule, write

$$\tilde{x} = x_0 + \rho\varepsilon, \qquad x_0 \sim p_{\text{data}}, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}), \quad \rho > 0. \tag{3}$$

Here ρ is a noise standard deviation, not a timestep. The conditional $q_\rho(\tilde{x} \mid x_0) = \mathcal{N}(x_0, \rho^2\mathbf{I})$ is known, but its marginal $q_\rho(\tilde{x})$ averages over all clean examples. Different ρ values therefore define different noisy data distributions. Larger noise levels help connect regions of image space; lower ones preserve finer structure. A noise-conditioned model learns information appropriate to each level rather than using the same score everywhere.

3.3. From denoising targets to noise prediction

Differentiating the known conditional Gaussian gives the target $\nabla_{\tilde{x}} \log q_\rho(\tilde{x} \mid x_0) = -\varepsilon/\rho$. Denoising score matching connects this computable conditional target to the unknown marginal score [14, 18]:

$$\nabla_{\tilde{x}} \log q_\rho(\tilde{x}) = -\frac{\mathbb{E}[\varepsilon \mid \tilde{x}, \rho]}{\rho}. \tag{4}$$

This identity is the essential bridge. Under squared-error training, the ideal predictor of a random target is its conditional expectation. Consequently, a noise predictor minimizing expected $\|\varepsilon - \varepsilon_\theta(\tilde{x}, \rho)\|_2^2$ implicitly supplies a score estimate $-\varepsilon_\theta(\tilde{x}, \rho)/\rho$. Targets are generated by us, yet their relationship to noisy images depends on the training data. A network cannot generally identify the exact noise draw from a noisy image alone; its ideal output averages over compatible clean examples. Appendix C.3 derives the identity and the squared-error result.

3.4. Why this can produce new images

Learning denoising across noise levels provides a family of statistical directions. A sampling procedure can start from a simple random state and use those directions while moving toward lower noise levels. It is not required to begin with a corrupted real image or retrace a training image's forward trajectory. The shared denoiser has learned from many possible images; the generated sample is determined by the starting randomness, learned model, and sampling transitions.

Table 1. Qualitative comparison of representative image generators. According to [3, 6, 16]; these are architectural tendencies, not universal performance rankings.

Family	Learning idea	Sampling	Main trade-off
GAN	Generator competes with a discriminator	Typically one generator pass	Fast synthesis; balancing the adversarial game can be difficult
Autoregressive	Predict each variable from earlier variables	Sequential pixel/token prediction	Explicit factorization; sequential generation can be costly
Diffusion	Predict corruption across noise levels	Repeated denoising of a whole state	Flexible conditioning; multiple network evaluations

This explains the conceptual link, but a score alone does not specify a complete finite-step generator. We must still choose a forward schedule, a reverse distribution, and a training objective. DDPM supplies these choices explicitly. Its signal-scaled Gaussian construction differs from Eq. (3); the corresponding score–noise relationship is derived in Appendix C.3.

4. DDPM: Formulation and Worked Examples

4.1. Forward noising and its schedule

For a vectorized image with D normalized scalar coordinates, DDPM fixes a Markov chain that gradually adds Gaussian noise [6, Sec. 2]:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t \mathbf{I}\Big), \tag{5}$$

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{j=1}^{t} \alpha_j, \quad \bar{\alpha}_0 = 1.$$

x_0 is a clean image, $x_t \in \mathbb{R}^D$ is a noisy state, and the integer $t \in \{1, \dots, T\}$ indexes noise level rather than seconds. The dimensionless variance $\beta_t \in (0, 1)$ and identity matrix $\mathbf{I}$ specify independent per-coordinate innovations. The *schedule* $(\beta_1, \dots, \beta_T)$ controls how quickly signal disappears. The entire forward marginal has a closed form:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \mathbf{I}). \tag{6}$$

Thus training can create a noisy example at any selected t without simulating preceding steps. The aggregate noise ε in this expression is distinct from an individual Markov-step innovation. For a sufficiently small $\bar{\alpha}_T$, the terminal distribution is close to standard Gaussian noise. A finite schedule does not make this equality exact. Figure 1 contrasts the fixed forward process and the learned reverse process.

4.2. Why DDPM uses the simplified loss

DDPM defines a reverse generative chain starting from $p(x_T) = \mathcal{N}(0, \mathbf{I})$. Its image likelihood marginalizes the intermediate states, making direct likelihood optimization difficult. The paper therefore uses a variational upper bound on negative log likelihood [6, Secs. 2–3]. For $t > 1$ and fixed reverse variance, each intermediate KL term reduces to a weighted noise-prediction error:

$$L_{t-1} = \mathbb{E}\big[w_t \|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2\big] + C, \tag{7}$$

where $w_t > 0$ is determined by the schedule and reverse variance, and C does not depend on denoiser parameters θ. Appendix C.4 gives the bound, posterior, and exact coefficient. The endpoint reconstruction term is handled separately in that derivation.

The simplified objective drops these timestep-dependent coefficients [6, Eq. 14]:

$$L_{\text{simple}} = \mathbb{E}_{t, x_0, \varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2\right]. \tag{8}$$

t is sampled uniformly from 1 to T, x_0 from training images, and ε independently from a standard Gaussian; x_t is formed by Eq. (6). The predictor returns D noise coordinates, and the squared Euclidean norm sums their squared errors. Every target is known because the training algorithm drew the noise. The same network is conditioned on t and shared across noise levels.

Why this is meaningful. As Section 3.3 explains, the ideal noise predictor estimates $\mathbb{E}[\varepsilon \mid x_t, t]$, which determines the marginal score. Predicting noise is therefore a parameterization of a learned denoising direction, not a claim that Gaussian noise is recognizable independently of image structure. Predicting x_0 or the reverse mean is also possible; the usefulness of this particular parameterization requires evidence.

Why simplification helps in the reported setting. Removing w_t changes the relative emphasis of noise levels. Under the original DDPM schedule, it reduces emphasis on low-noise reconstruction relative to the variational objective. Ho et al. report better sample quality with this reweighting, while the original bound gives better likelihood-based code-lengths [6, Secs. 3.4, 4.1–4.2]. Section 6.2 presents their objective comparison. This is empirical support for a quality trade-off, not a theorem that unweighted MSE is universally optimal.

What efficiency means here. One training update samples x_0, t, ε, constructs x_t, evaluates the network once, and

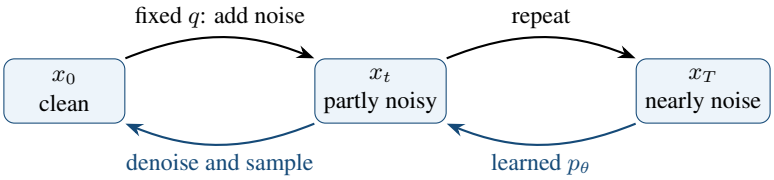


Figure 1. Original schematic of DDPM's two directions, based on [6]. Reverse arrows represent a learned distribution, not exact inversion of an individual forward trajectory.

backpropagates its MSE. There is no need to unroll all T denoising steps in each update. This practicality follows from the closed-form marginal and stochastic timestep sampling, not solely from removing w_t. The simplified loss does not itself reduce the number of sampling steps or the spatial cost of a denoiser evaluation.

4.3. Reverse denoising and generation

DDPM models a reverse transition as

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t), \sigma_t^2 \mathbf{I}\big)\,. \tag{9}$$

Here μ_θ is computed from the noise predictor; $\sigma_t \geq 0$ is a chosen reverse standard deviation. For example, the paper considers $\sigma_t^2 = \beta_t$ for intermediate steps. Its epsilon-parameterized sampling update is [6, Eq. 11, Algorithm 2]

$$x_{t-1} = \underbrace{\frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon_\theta(x_t, t)\right)}_{\mu_\theta(x_t, t)} + \sigma_t u, \qquad u \sim \mathcal{N}(0, \mathbf{I}). \tag{10}$$

Starting with $x_T \sim \mathcal{N}(0, \mathbf{I})$, run this update for $t = T, T-1, \ldots, 1$; the final step omits the fresh noise term. Predicted noise determines the reverse *mean*; u is newly drawn sampling noise. The algorithm does not simply subtract a predicted noise vector from the current state.

The true reverse conditional need not be Gaussian. Small forward steps motivate DDPM's Gaussian approximation; Appendix C.5 connects its mean to the score explicitly. Many local transitions allow a flexible overall distribution. A one-step deterministic MSE estimate from almost pure noise would instead approach an average of compatible images, not provide a general sampling rule. This does not rule out other models or trained few-step samplers; it explains the original DDPM design.

DDPM's contribution is the combination of effective noise-prediction training, its connection to denoising score matching, and demonstrated image synthesis. It builds on the earlier diffusion construction rather than introducing the entire family from scratch.

4.4. A complete one-dimensional calculation

For one step $t = 1$, choose normalized $x_0 = 0.8$, variance $\beta_1 = 0.1$, and Gaussian realization $\varepsilon = -0.5$. This larger variance makes the arithmetic visible; the image schedule below uses smaller steps. Equation (5) gives

$$\begin{aligned} \alpha_1 &= 1 - 0.1 = 0.9, \\ \sqrt{\alpha_1}\, x_0 &= \sqrt{0.9}\,(0.8) = 0.758947, \\ \sqrt{\beta_1}\, \varepsilon &= \sqrt{0.1}\,(-0.5) = -0.158114, \\ x_1 &= 0.758947 - 0.158114 = \boxed{0.600833}. \end{aligned}$$

Thus $q(x_1 \mid x_0 = 0.8) = \mathcal{N}(0.758947, 0.1)$: its mean is 0.758947, variance 0.1, and standard deviation $\sqrt{0.1} = 0.316228$. Across independent draws, $\mathbb{E}[x_1 \mid x_0] = \sqrt{0.9}\, x_0$ since $\mathbb{E}[\varepsilon] = 0$. The realization 0.600833 differs from that expectation; the conditional variance is β_1.

4.5. Five noise levels on an original small image

Our original 28×28 geometric house has foreground $+1$ and background -1, without an external dataset. The script `code/make_figures.py` simulates one Markov trajectory using Eq. (5) with independent noise at every step. It uses NumPy's default generator, seed 420, $T = 1000$, and the linear schedule $\beta_t = 10^{-4} + (t-1)(0.02 - 10^{-4})/999$, following DDPM's schedule choice [6, Sec. 4].

The last column is a schedule-based signal-to-noise coefficient ratio, also a per-coordinate variance ratio for unit-variance data. This illustrates corruption, not learned denoising or generation.

5. LDM: Latent Space and Efficient Generation

5.1. Why move from pixels to a latent representation?

DDPM supplies a way to generate images, but repeatedly processing a high-resolution pixel grid is expensive. LDM separates learning a useful image representation from learning its generative distribution [10, Secs. 3.1–3.2]. Its first stage compresses images; its second stage applies diffusion to the resulting spatial tensors. The central question is how much computation can be removed while preserving information needed for convincing reconstruction.

A latent is a learned feature representation, not a smaller JPEG or a text description. For RGB input $x_0 \in \mathbb{R}^{H \times W \times 3}$, let $\tilde{z} \in \mathbb{R}^{h \times w \times c}$, where $h = H/f$, $w = W/f$, $f \geq 1$ is the



Figure 2. Original forward-noising illustration at five timesteps. The same trajectory is shown throughout. Values are clipped to $[-1, 1]$ only for display; stored arrays and calculations remain unclipped. The equation and schedule follow [6].

Table 2. Coefficients computed by the supplied script. Signal and noise columns refer to Eq. (6), not measured image quality.

t	$\bar{\alpha}_t$	Signal $\sqrt{\bar{\alpha}_t}$	Noise SD $\sqrt{1-\bar{\alpha}_t}$	$\bar{\alpha}_t/(1-\bar{\alpha}_t)$
0	1.000000	1.000000	0.000000	∞
50	0.971016	0.985401	0.170248	33.5015
200	0.659039	0.811812	0.583919	1.9329
500	0.078587	0.280334	0.959902	0.0853
1000	0.000040	0.006353	0.999980	0.000040

spatial downsampling factor, and c is the latent channel count. For example, an LDM configuration with $f = 4, c = 3$ maps $256 \times 256 \times 3$ to $64 \times 64 \times 3$ [10, Table 8]. This reduces the number of spatial positions by 16, but the end-to-end runtime and memory depend on more than the state shape.

5.2. VAE encoder, decoder, and the compression trade-off

We emphasize LDM's KL-regularized, VAE-like autoencoder. Its encoder predicts a distribution over representations of an input image [8, 10]:

$$\begin{aligned} q_\phi(\tilde{z} \mid x_0) &= \mathcal{N}\big(\mu_\phi(x_0), \mathrm{diag}(\sigma_\phi^2(x_0))\big)\,, \\ \tilde{z} &= \mu_\phi(x_0) + \sigma_\phi(x_0) \odot \xi, \quad \xi \sim \mathcal{N}(0, \mathbf{I}), \\ \hat{x}_0 &= D_\psi(\tilde{z}). \end{aligned} \tag{11}$$

ϕ, ψ are encoder and decoder parameters. The encoder outputs latent means μ_ϕ and positive standard deviations σ_ϕ; diag forms a diagonal covariance and $\odot$ denotes elementwise multiplication. The decoder D_ψ reconstructs an image $\hat{x}_0$. Reparameterization places randomness in ξ, enabling gradient-based training. This posterior noise is different from the diffusion noise added in the second stage.

The encoder must retain spatial and perceptual information; the decoder must learn how to turn it back into image detail. Reconstruction quality therefore limits what the complete generator can preserve. Excessive compression may erase fine text, small objects, or textures that later diffusion cannot faithfully restore.

Regularization makes the representation suitable for modeling, but the first stage is not merely a textbook VAE with pixel reconstruction loss. LDM uses perceptual and patch-adversarial reconstruction objectives with a weak KL penalty [10, Sec. 3.1]. Perceptual loss compares visual features; adversarial training encourages locally realistic texture. A textbook VAE objective is given in Appendix D.1 to distinguish these choices. The original paper also studies VQ regularization, so LDM is broader than this KL/VAE variant. Neither weak KL regularization nor successful reconstruction makes clean encoded latents exactly standard normal. The diffusion prior must still learn their distribution.

5.3. Diffusion training and sampling in latent space

After the autoencoder is trained, its parameters are fixed while the latent denoiser is trained. Let $z_0 = a\tilde{z}$, with optional fixed scale $a > 0$ to match latent magnitudes to the noise schedule; $a = 1$ means no rescaling. Diffusion acts on $d_z = hwc$ latent coordinates:

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon_z, \quad \varepsilon_z \sim \mathcal{N}(0, \mathbf{I}_{d_z}), \tag{12}$$

$$L_{\mathrm{LDM}} = \mathbb{E}_{x_0, y, t, \xi, \varepsilon_z}\left[\|\varepsilon_z - \varepsilon_\theta(z_t, t, \tau_\eta(y))\|_2^2\right]. \tag{13}$$

The condition y may be a text prompt, encoded by τ_η with parameters η. The time-conditioned denoiser, typically a U-Net, returns a tensor of the same shape as z_t. The conditional objective follows LDM [10, Eq. 3], here making stochastic encoding explicit. Posterior-mean encoding omits ξ.

For generation, start with latent Gaussian noise and repeatedly apply a compatible reverse sampler using the condi-

tional predictor. Decode the final latent once as $D_\psi(z_0/a)$, undoing the training scale. Pure text-to-image generation needs no real input image or image encoder. The encoder is needed to form training representations, whereas the decoder converts a generated representation into pixels. Figure 3 makes these roles explicit.

5.4. Conditioning and the complete pipeline

LDM uses cross-attention to make denoising depend on a condition such as text [10, Sec. 3.3]. Spatial image features supply queries; prompt-token embeddings supply keys and values. Learned attention weights retrieve text information useful for the current features. Thus a prompt changes the denoising predictions throughout the sampling loop; it is not pasted onto the decoded image. The matrix formulation and an optional toy calculation are confined to Appendix D.2.

Classifier-free guidance can strengthen conditional influence by using $\varepsilon_u + s(\varepsilon_c - \varepsilon_u)$, where $\varepsilon_u, \varepsilon_c$ are unconditional and conditional predictions and $s \geq 0$ is the guidance scale [5]. Here $s = 1$ is ordinary conditional sampling. Stronger guidance can trade diversity for conditional fidelity; it does not guarantee correct counting, spatial relations, or attribute binding. Standard CFG uses two denoiser evaluations per step. Details are in Appendix D.3.

The two papers therefore address complementary questions: DDPM learns how iterative denoising can generate an image distribution; LDM learns a representation in which that iteration is less expensive. Compression reduces repeated spatial work. Faster samplers such as DDIM instead reduce the number of evaluations [13]; the two ideas can be combined without treating them as the same speedup.

6. Results and Discussion

6.1. Original illustrations and evidence boundaries

Our own results are the scalar calculation in Section 4.4 and the forward trajectory in Section 4.5. Figure 2 shows the synthetic house losing recognizable structure as the signal multiplier falls from 1 to 0.00635. Table 2 relates the sequence to the schedule. One random trajectory, especially with display clipping, does not establish a population-level image-quality trend.

These calculations illustrate forward corruption, not successful learned reversal. No pretrained generation, denoising benchmark, GPU runtime, peak-memory, or prompt-adherence measurement was performed. The following two comparisons are published results from different studies and tasks; they must not be combined into a DDPM-versus-LDM quality ranking.

Table 3. Published DDPM objective comparison [6, Table 1]. NLL upper bounds are in bits/dimension; lower values are better. These are not measurements from our team.

Objective	FID	NLL bound
Variational bound L	13.51	3.70
Simplified L_{simple}	3.17	3.75

6.2. Published DDPM evidence for the objective choice

Table 3 reports Ho et al.'s unconditional CIFAR-10 comparison with epsilon prediction and fixed isotropic reverse variance [6, Tables 1–2]. The simplified objective lowers FID from 13.51 to 3.17, while its reported negative-log-likelihood upper bound is slightly higher. This demonstrates a sample-quality/codelength trade-off in the authors' setting, rather than equivalent optimization of the original likelihood bound.

FID compares generated and reference feature-distribution statistics [4]; it does not directly measure every image's quality or prompt adherence. DDPM computes these CIFAR-10 FIDs against training images; its reported test-reference FID for the simplified model is 5.24 [6, Sec. 4.1]. Stating the reference set matters. The ablation supports the combination of epsilon prediction and the simplified objective, not a claim that any unweighted reconstruction loss improves any generative model.

6.3. A published pixel-versus-latent setting

We select the *Places inpainting* comparison in Rombach et al. [10, Sec. 4.5, Table 6]: pixel-space LDM-1 versus LDM-4 with a KL-regularized first stage and attention. Their denoisers have a matched parameter count. The published protocol trains on 256×256 crops; the supplementary protocol specifies fixed validation masks and a 2,000-image validation set. We preserve the table's two sampling resolutions and six-epoch endpoint.

Our arithmetic gives sampling-throughput ratios $0.97/0.26 = 3.73$ at 256^2 and $0.34/0.07 = 4.86$ at 512^2. The epoch-time ratio is $20.66/7.66 = 2.70$ and training-throughput ratio $0.32/0.11 = 2.91$. The reported FID is 9.53 points lower, a relative decrease of 38.5%; this percentage is *not* a percentage increase in perceptual quality.

FID (Fréchet Inception Distance) compares feature-distribution statistics between generated and reference images [4]. It does not directly measure prompt adherence, fairness, factual correctness, or the quality of every individual image. The result supports an efficiency–quality advantage for this configuration and training endpoint. Six equal epochs do not imply equal wall-clock training budgets or equal convergence. Moreover, the table flags GPU-setting/batch-size differences from the paper's Figure 7; its ratios should not

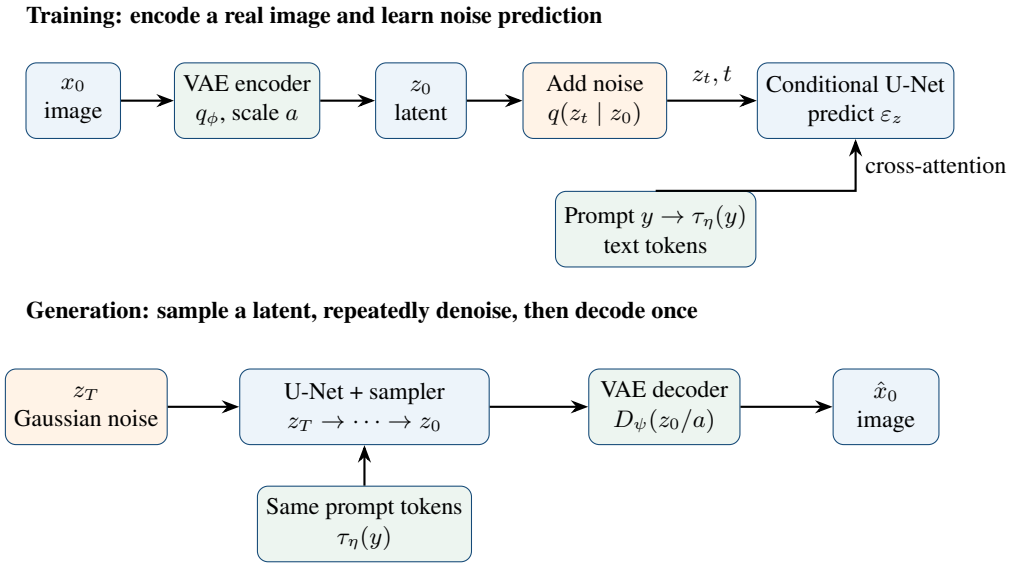


Figure 3. Original latent diffusion pipeline, based on [10, Fig. 3 and Sec. 3]. The VAE is learned first and frozen for diffusion training; its decoder runs after the denoising loop. Pure text-to-image generation does not require a real input image or the image encoder.

Table 4. Published inpainting comparison, transcribed from [10, Table 6]. These are the paper's measurements, not a benchmark run by our team. Higher throughput and lower FID are better.

Criterion	Pixel LDM-1	LDM-4 (KL, with attention)
Training throughput (samples/s)	0.11	0.32
Sampling throughput at 256^2 (samples/s)	0.26	0.97
Sampling throughput at 512^2 (samples/s)	0.07	0.34
Training + validation (hours/epoch)	20.66	7.66
Validation FID after epoch 6 (2k samples)	24.74	15.21
Peak memory (GB)	Not reported in this table	

be universalized. Separate autoencoder pretraining is an additional cost not isolated by this table. Inpainting uses spatial conditions, so this experiment does not itself establish a text-prompt-control advantage.

6.4. Speed, quality, memory, and control: explicit criteria

Table 5 connects the benchmark to the mechanisms developed in Section 5. Its memory and control entries are analytical comparisons, not unreported empirical measurements.

Where speed comes from. An analytical latency decomposition is

$$\mathcal{T}_{\text{gen}} \approx \mathcal{T}_{\text{text}} + K\, r\, \mathcal{T}_{\text{U-Net}} + \mathcal{T}_{\text{decode}}, \tag{14}$$

where every $\mathcal{T}$ is measured in seconds, K is the number of sampling updates, and r is the effective number of denoiser evaluations per update (1 without standard CFG, 2 with it, before batching effects). The final term is absent in pixel diffusion. Latent compression mainly reduces the cost of a U-Net evaluation; DDIM can reduce K [13]. Fewer updates, higher guidance, and greater compression are separate design choices with different quality consequences.

Why a smaller state is not a guaranteed memory ratio. At 256^2, RGB has 196,608 coordinates; the illustrative $64 \times 64 \times 3$ latent has 12,288, giving a 16:1 state-element ratio. Generally it is $3f^2/c$. At fixed feature width, spatial activation storage also benefits from fewer positions, but model weights, attention, batching, and the full-resolution decoder affect the peak. Therefore neither a $16\times$ runtime speedup nor a $16\times$ total-memory reduction follows from this arithmetic.

Quality and applicability. Moderate compression can make iterative generation practical at larger resolutions. Tasks needing exact text, tiny structures, or pixel-faithful reconstruction must also inspect autoencoder error. Stronger guidance and lower FID cannot substitute for checking requested object attributes and relations. These distinctions explain why the favorable inpainting result is useful evidence without settling every application.

Table 5. Pixel versus latent diffusion. The concrete latent setting is LDM-4 with KL regularization. Architecture-based analysis is informed by [5, 6, 10]; measured values are confined to Table 4.

Criterion	Pixel diffusion	Latent diffusion (KL, $f = 4$)
Speed	Repeatedly processes a pixel-resolution state	Smaller repeated states; one final decode. The published inpainting throughput is higher in the selected setting
Quality	No separate autoencoder bottleneck; still limited by denoiser, data, and sampling	Compression can retain useful structure but lose detail or introduce reconstruction artifacts
Memory	Larger spatial states and potentially larger feature maps	Fewer latent positions; autoencoder/text-encoder weights and decoder activations add memory. End-to-end peak memory needs measurement
Control	Can support text conditioning and guidance when trained for it	Cross-attention and guidance support flexible prompts; compression itself does not guarantee better semantic control

6.5. Training data, bias, copyright, and limitations

Data dependence. The VAE learns what can be represented, the denoiser learns what is likely, and the text encoder learns which language features connect to imagery. Errors or underrepresentation at any stage can propagate to outputs. Large image–text corpora such as LAION-5B illustrate the scale of this dependency [11]; they are not interchangeable with the specific datasets used in every original LDM experiment. Caption noise, duplicate images, language imbalance, and missing provenance are distinct concerns. Our analytical recommendation is to document dataset versions, sources, filters, and evaluation splits, and audit duplicates and underrepresented concepts.

Bias and prompt control. Bianchi et al. [1] report demographic stereotype amplification in text-to-image systems, including prompts that do not explicitly name demographic groups. Latent compression does not remove these associations. Because guidance amplifies conditional preferences and can reduce diversity, stronger prompt adherence should not be assumed to imply fairer representation. Evaluation should compare distributions across multiple prompts, groups, and seeds; prompt rewriting alone is insufficient evidence of mitigation.

Memorization and copyright. Carlini et al. [2] demonstrate extraction of memorized training examples from diffusion models. This establishes a possible failure mode, not that every output is copied. Dataset availability, permission to use underlying works, memorization risk, and rights in an output are separate questions. The U.S. Copyright Office's 2025 pre-publication analysis distinguishes data collection, training, outputs, and licensing [15]; it is a U.S. policy source, not Vietnamese law. Compression is no technical guarantee against reproduction. Our recommendations are documented provenance, appropriate permissions, deduplication, and similarity checks for sensitive use cases; none alone certifies legal compliance.

Limits of this report. Our original evidence covers a scalar calculation and one synthetic forward trajectory. The model evidence comprises published DDPM objective results and an LDM inpainting comparison, not our own matched text-to-image benchmark. We have no measured peak-memory values or prompt-adherence scores. A future study should hold hardware, precision, batch size, sampler, step count, guidance, resolution, and dataset fixed, report both autoencoder and denoiser costs, and evaluate quality and control separately.

7. Conclusion

Learning to remove noise enables generation because denoising targets carry information about noisy data distributions. At the squared-error optimum, noise prediction determines a marginal score; a reverse sampling process uses that information to move a new random state toward the learned image distribution. DDPM makes this connection concrete through Gaussian transitions, a tractable training procedure, and an empirically effective simplified objective. The quality benefit of that objective must be distinguished from likelihood optimization and sampling speed.

LDM preserves the denoising principle while moving repeated computation into a learned latent space. Its encoder/decoder determine what can be compressed and reconstructed; the diffusion model learns how to generate those representations. Published inpainting evidence supports reduced computational cost in a specific latent setting, with autoencoder costs and reconstruction limits still relevant. Together, DDPM and LDM answer the guiding question at the levels of generative mechanism and practical representation, without making efficiency a guarantee of semantic control, fairness, or unrestricted data use.

References

[1] Federico Bianchi, Pratyusha Kalluri, Esin Durmus, Faisal Ladhak, Myra Cheng, Debora Nozza, Tatsunori Hashimoto, Dan Jurafsky, James Zou, and Aylin Caliskan. Easily accessible text-to-image generation amplifies demographic stereotypes at large scale. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency*, 2023.

[2] Nicholas Carlini, Jamie Hayes, Milad Nasr, Matthew Jagielski, Vikash Sehwag, Florian Tramèr, Borja Balle, Daphne Ippolito, and Eric Wallace. Extracting training data from diffusion models. In *32nd USENIX Security Symposium*, 2023.

[3] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014.

[4] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. In *Advances in Neural Information Processing Systems*, 2017.

[5] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022. A short version appeared at a NeurIPS 2021 workshop.

[6] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, pages 6840–6851, 2020.

[7] IEEE. ICCV 2025 Author Kit, 2025. Official author kit; style and bibliography files distributed with the project.

[8] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014.

[9] OpenAI. Codex, 2026. AI assistance for drafting, LaTeX, code, and source checking; see Appendix B.

[10] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. arXiv v2, April 2022; includes supplementary material.

[11] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, et al. LAION-5B: An open large-scale dataset for training next generation image-text models. In *Advances in Neural Information Processing Systems*, 2022.

[12] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 2256–2265, 2015.

[13] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.

[14] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems*, 2019.

[15] U.S. Copyright Office. Copyright and artificial intelligence, part 3: Generative AI training. Technical report, U.S. Copyright Office, 2025. Pre-publication version; consulted November 5, 2026. A U.S. policy analysis, not a statement of Vietnamese law.

[16] Aäron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1747–1756, 2016.

[17] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[18] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

A. Requirement-to-Evidence Map

This map concerns the written report. The assignment's separate slide, oral-presentation, balanced-participation, and Q&A requirements require presentation deliverables and cannot be fulfilled by a report alone. The main text occupies pages 1–8; references and appendices are separate.

Table 6. Coverage of the supplied CS420 S06 seminar brief.

Task	Required content	Location / evidence
1	Generative versus discriminative; GAN, autoregressive, diffusion	Sections 2.1, 2.2; Table 1
2	Forward process, beta schedule, Gaussian noise, noise predictor, reverse sampling; both-direction diagram	Section 4; Eqs. (5)–(10); Figure 1
3	Manual 1D forward step, symbols, stochastic expectation	Section 4.4; $x_1 = 0.600833$; conditional mean and variance
4	Small image at five noise levels; optional pretrained demonstration	Section 4.5; Figure 2; Table 2; executable code and arrays. Pretrained generation was not performed
5	VAE encoder/decoder, latent operations, cross-attention, pipeline	Sections 5.2–5.4; Eqs. (11)–(13); Figure 3
6	Speed, quality, prompt control, data, copyright, bias; explicit pixel/latent comparison	Sections 5.4, 6.3–6.5; Tables 4–5

Required representations. Equation (5) is the Gaussian forward transition; Eq. (8) is the simplified noise-prediction objective. The worked example gives a complete substitution, result, and interpretation. Appendix C records ranges, units, and modeling assumptions.

Paper reading and synthesis. DDPM's forward/reverse method and contribution are reviewed in Section 4. LDM's representation, method, conditioning, contributions, and differences are developed in Section 5. Sections 6.2 and 6.3 identify the source tables for the DDPM objective and LDM inpainting comparisons. The report includes Abstract, Introduction, Background/Paper Review, Method/Worked Example, Results and Discussion, Conclusion, and References. English is used according to the commissioning instruction, which overrides the brief's Vietnamese-language request.

B. Reproducibility, Evaluation Protocol, and Attribution

B.1. Reproduce the original illustration

The Overleaf project already contains the generated figure; Python is not needed to compile the report. To regenerate it locally:

```
python -m pip install -r code/requirements.txt
python code/make_figures.py
latexmk -pdf -interaction=nonstopmode -halt-on-
    error main.tex
```

The figure uses a synthetic geometric image, not MNIST or another dataset. The script records the seed, schedule coefficients, library versions, scalar example, and benchmark-ratio arithmetic in `figures/analysis.json`. It saves unclipped frames, timesteps, the schedule, and cumulative coefficients in `noise_arrays.npz`, and summary coefficients in `noise_levels.csv`. The delivered run used NumPy 2.3.5 and ReportLab 4.4.9. The sequence is one sampled Markov path; identical seeds and library versions reproduce it. No image-generation checkpoint is involved.

B.2. A future prompt and runtime study (not executed)

Select and record exact checkpoint identifiers, revisions, dataset licenses, hardware, software versions, numerical precision, resolution, and batch size. Keep each fixed within a comparison and record warm-up policy. For latent models, measure text encoding, denoising, decoding, and end-to-end latency separately, and record peak allocated memory over the entire pipeline.

Use paired prompts such as *a red cube to the left of a blue sphere* and *a green cube to the left of a blue sphere*; separately swap the spatial relation. Predefine at least 20 seeds per condition and inspect all samples. Vary sampling steps and guidance in separate sweeps. Report object presence, color binding, relation correctness, and diversity; do not select only attractive examples. This is a proposed evaluation design, not empirical evidence of a model's prompt capability.

B.3. Provenance and AI assistance

Figures 1, 3, and 4 are original TikZ schematics explaining cited methods; no paper figure is reproduced. Figure 2 and Table 2 are generated by the supplied code. Table 3 reproduces selected DDPM Table 1 values. Table 4 is transcribed from LDM Table 6; ratios are our arithmetic. The two qualitative comparison tables are analytical syntheses with sources in their captions.

The document uses the requested ICCV 2025 author kit [7], with its official style and bibliography files. It uses 10-point type, two columns, and US Letter paper, following the new template request. The author block retains HCMUS and CS420 information and three name/ID placeholders. Template provenance is included in the project.

OpenAI Codex [9] assisted with drafting, equations, code, diagram construction, source checking, and LaTeX preparation. Preparation checks included consulting the original DDPM/LDM equations and DDPM Tables 1–2 and LDM Table 6, recalculating the numerical examples and ratios, and compiling and visually reviewing the PDF. The named student authors should review and accept the final manuscript and replace the author placeholders before submission. No human study, new trained model, or pretrained generation benchmark is claimed.

C. Notation, Assumptions, and Additional Derivation

C.1. Ranges and units

Images are normalized and dimensionless. Clean pixels in the illustration lie in $[-1, 1]$; noisy states, latent values, and predicted noise are real-valued and need not stay within that interval. There is no physical time unit in the diffusion index.

Table 7. Notation used in the equations; meanings are also introduced beside the corresponding equations in the main text.

Symbols	Meaning, range, and units
$x_0, x_t, \hat{x}_0$	Clean/noisy/reconstructed image; vector in $\mathbb{R}^D$ or an equivalent image tensor; normalized intensity units
D, H, W, h, w, c, d_z	Positive integer dimensions. $D = 3HW$ for RGB, $d_z = hwc$; H, W count pixels, h, w count latent positions
t, T, j	Integer timestep, terminal step, and product index; $1 \le t \le T$, $1 \le j \le t$
$\beta_t, \alpha_t, \bar{\alpha}_t$	Dimensionless schedule coefficients; $0 < \beta_t < 1$, $\alpha_t = 1 - \beta_t$, $0 < \bar{\alpha}_t \le 1$
$q, p_\theta, p_{\text{data}}$	Fixed forward transition, learned reverse/generative distribution, and data distribution, respectively
$\rho, \tilde{x}, q_\rho, s$	Positive Gaussian perturbation SD, perturbed image, noisy marginal, and score in Section 3; s there is a vector field, distinct from the scalar CFG scale
$w_t, \tilde{\beta}_t, \tilde{\mu}_t$	Positive loss weight; forward-posterior variance and mean, for $t > 1$; see Appendix C.4
$\mathcal{N}(\mu, \Sigma), \mathbf{I}$	Gaussian with mean μ and covariance Σ; identity covariance of the relevant dimension
$\varepsilon, \varepsilon_z, u, \xi$	Independent standard-normal draws in their respective sampling operations; each coordinate has mean 0 and variance 1
$\sigma_t, \mu_\phi, \sigma_\phi$	Reverse noise standard deviation $\sigma_t \ge 0$; VAE posterior mean and positive standard deviations
$\theta, \phi, \psi, \eta, \omega$	Trainable parameter arrays of denoiser, VAE encoder, decoder, condition encoder, and illustrative classifier
$\tilde{z}, z_0, z_t, f, a$	Raw/clean-scaled/noisy latent; spatial factor $f \ge 1$, with H, W divisible by f; fixed dimensionless scale $a > 0$
$y, \tau_\eta(y), F, C$	Condition, encoded condition, U-Net features, and token embedding matrix. Continuous embeddings are real-valued
n, m, d_u, d_y, d_a, d_v	Positive integer counts and widths for cross-attention; see Appendix D.2 for all matrix shapes
Q, K, V, A, O	Query, key, value, attention-weight, and output matrices; $A_{ij} \in [0, 1]$ and $\sum_j A_{ij} = 1$
$s, w, p_{\text{drop}}, \varnothing$	CFG scale $s \ge 0$, alternative scale $w = s - 1$, dropout probability in $(0, 1)$, and null condition
$K, r, \mathcal{T}$	In the cost model only, K counts updates (not attention keys), r counts evaluations/update, and $\mathcal{T}$ is seconds
$L_{\text{simple}}, L_{\text{LDM}}$	Expected summed squared noise-prediction errors; dimensionless squared normalized-coordinate units
$\mathbb{E}, \lVert \cdot \rVert_2, \mathrm{D}_{\text{KL}}$	Expectation, Euclidean norm, and KL divergence; natural logarithms make KL and negative log likelihood use nats

C.2. Why the forward marginal is available in closed form

Assume independent standard-normal innovations $\varepsilon^{(1)}, \varepsilon^{(2)}$ and let x_0 be fixed. Applying Eq. (5) twice gives

$$x_1 = \sqrt{\alpha_1}x_0 + \sqrt{\beta_1}\varepsilon^{(1)},$$
$$x_2 = \sqrt{\alpha_2\alpha_1}x_0 + \sqrt{\alpha_2\beta_1}\varepsilon^{(1)} + \sqrt{\beta_2}\varepsilon^{(2)}.$$

The two independent noise terms have total variance $\alpha_2\beta_1 + \beta_2 = \alpha_2(1-\alpha_1) + (1-\alpha_2) = 1 - \alpha_2\alpha_1$. A sum of independent Gaussians is Gaussian. Repeating this argument yields Eq. (6), with mean $\sqrt{\bar{\alpha}_t}x_0$ and covariance $(1-\bar{\alpha}_t)\mathbf{I}$ [6].

Gaussian innovations are independent across coordinates and steps, but the noisy images themselves have correlations inherited from x_0. When $\bar{\alpha}_T$ is close to zero, dependence on the starting image is weak and a standard-normal starting prior is a useful approximation. A finite schedule need not make it exactly equal to that prior. The learned Gaussian reverse transition is a modeling approximation motivated by small forward steps; no finite network is assumed to know the exact reverse conditional.

C.3. Denoising score matching and the MSE optimum

Fix a nonzero noise level and write $x_t = a_t x_0 + b_t\varepsilon$ with $a_t, b_t > 0$, $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ independent of x_0. The additive example of Section 3 uses $a_t = 1, b_t = \rho$; DDPM uses $a_t = \sqrt{\bar{\alpha}_t}$ and $b_t = \sqrt{1-\bar{\alpha}_t}$. These a_t coefficients are unrelated to the fixed latent scale a. Let q_t be the marginal over noisy data. Gaussian smoothing makes it positive and smooth; assume sufficient integrability to exchange integration and differentiation. Then

$$q_t(x_t) = \int p_{\text{data}}(x_0)q(x_t \mid x_0)\,dx_0,$$
$$\nabla \log q_t(x_t) = \int q(x_0 \mid x_t, t)\nabla \log q(x_t \mid x_0)\,dx_0. \quad (15)$$

The conditional Gaussian has score

$$\nabla \log q(x_t \mid x_0) = -\frac{x_t - a_t x_0}{b_t^2} = -\frac{\varepsilon}{b_t}. \quad (16)$$

Taking its posterior expectation in Eq. (15) gives

$$\nabla \log q_t(x_t) = -\frac{\mathbb{E}[\varepsilon \mid x_t, t]}{b_t}. \quad (17)$$

This is the conditional-to-marginal identity underlying denoising score matching [14, 18].

For any vector-valued predictor g and finite second-moment target Y, condition on an input X. With $m(X) = \mathbb{E}[Y \mid X]$, expand

$$\mathbb{E}[\|Y - g(X)\|^2 \mid X] = \mathbb{E}[\|Y - m(X)\|^2 \mid X] + \|m(X) - g(X)\|^2. \quad (18)$$

The cross term vanishes because $\mathbb{E}[Y - m(X) \mid X] = 0$. Thus the ideal MSE predictor is $m(X)$. Setting $Y = \varepsilon$ and $X = (x_t, t)$ justifies the interpretation in Section 3.3. A finite trained network only approximates that optimum.

If $\varepsilon_\theta = -b_t s_\theta$, a conditional-target score loss satisfies

$$\mathbb{E}\|s_\theta + \varepsilon/b_t\|^2 = \frac{1}{b_t^2}\mathbb{E}\|\varepsilon - \varepsilon_\theta\|^2 \quad (19)$$

at a fixed t. Across timesteps, weights must be accounted for: unweighted noise MSE corresponds to weighting this score loss by b_t^2. The two unweighted losses should not be identified without that factor. The identity is used only for $b_t > 0$, not at the clean endpoint.

C.4. From the variational bound to weighted noise MSE

DDPM defines the joint generator and fixed forward trajectory [6, Secs. 2–3] as

$$p_\theta(x_{0:T}) = p(x_T)\prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t),$$
$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1}). \quad (20)$$

Jensen's inequality bounds negative log likelihood by $L = \mathbb{E}_q[\log q(x_{1:T} \mid x_0) - \log p_\theta(x_{0:T})]$. Rearranging gives

$$L = \mathbb{E}\Bigg[\mathrm{D}_{\mathrm{KL}}(q(x_T \mid x_0)\|p(x_T)) + \sum_{t=2}^{T}\mathrm{D}_{\mathrm{KL}}\left(q(x_{t-1} \mid x_t, x_0)\|p_\theta(x_{t-1} \mid x_t)\right) - \log p_\theta(x_0 \mid x_1)\Bigg]. \quad (21)$$

The terminal term is fixed when the schedule and prior are fixed. The endpoint term is a reconstruction/decoder likelihood; DDPM's original implementation uses a discretized decoder for image integers. It is not simply another intermediate Gaussian KL.

For $t > 1$, conditioning on x_0 yields a Gaussian posterior with

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t,$$
$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\beta_t}{1-\bar{\alpha}_t}x_0 + \frac{\sqrt{\alpha_t}(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}x_t. \quad (22)$$

With fixed reverse variance $\sigma_t^2 > 0$, the Gaussian KL is $\mathbb{E}[\|\tilde{\mu}_t - \mu_\theta\|^2/(2\sigma_t^2)] + C$. Substituting $x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\varepsilon)/\sqrt{\bar{\alpha}_t}$ gives

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\varepsilon\right). \quad (23)$$

Replacing ε by ε_θ defines the model mean in Eq. (10); subtracting the two means gives

$$w_t = \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)}. \tag{24}$$

These are DDPM's Eqs. 8–12. The simplified objective omits this coefficient for intermediate steps; its $t = 1$ term is related to the endpoint decoder using the approximation explained in DDPM Section 3.4. Consequently L_{simple} is a reweighted surrogate, not numerically the original variational bound. The CIFAR-10 results in Section 6.2 empirically motivate that change.

C.5. Why the score determines the reverse mean

Apply the marginal-score identity to a single Gaussian forward step:

$$\nabla \log q_t(x_t) = -\frac{x_t - \sqrt{\alpha_t}\mathbb{E}[x_{t-1} \mid x_t, t]}{\beta_t}. \tag{25}$$

Therefore its true reverse conditional mean is

$$\mathbb{E}[x_{t-1} \mid x_t, t] = \frac{x_t + \beta_t \nabla \log q_t(x_t)}{\sqrt{\alpha_t}}. \tag{26}$$

Using the estimated DDPM score $-\varepsilon_\theta(x_t, t)/\sqrt{1-\bar{\alpha}_t}$ gives the reverse mean in Eq. (10). The conditional-mean identity is exact under the stated assumptions, but it does not make the full reverse conditional Gaussian. Its higher moments and possible multimodality are not specified by the mean alone. DDPM uses a Gaussian approximation motivated by small forward steps and learns an approximate score.

C.6. Why noise prediction is useful without knowing the true noise

For a fixed x_t and t, the vector minimizing expected squared prediction error is $\mathbb{E}[\varepsilon \mid x_t, t]$ (and, in a conditional model, $\mathbb{E}[\varepsilon \mid x_t, t, y]$). Thus a trained predictor estimates a statistical direction shared by plausible clean examples, not an oracle that recovers every random draw. Its corresponding clean-state estimate is

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{27}$$

This follows by rearranging Eq. (6); prediction errors remain and can be magnified at small $\bar{\alpha}_t$. The reverse sampler uses predictions over many levels rather than treating one estimate as guaranteed recovery. The same argument applies in latent space, followed by the learned decoder.

C.7. Two training stages and their different roles

Stage one updates encoder ϕ and decoder ψ to preserve useful image information in a regularized representation. Stage two fixes them and trains denoiser θ with Eq. (13). The original LDM conditional formulation jointly optimizes the condition encoder and denoiser [10, Sec. 3.3]; particular later implementations may instead keep the text encoder fixed.

The distinction matters: improving the denoiser cannot guarantee exact recovery of details that the first stage systematically discards. Conversely, a high-quality autoencoder does not itself provide the conditional latent distribution needed for generation. The VAE prior regularizer and the learned diffusion prior play different roles.

D. Supporting Autoencoder and Conditioning Details

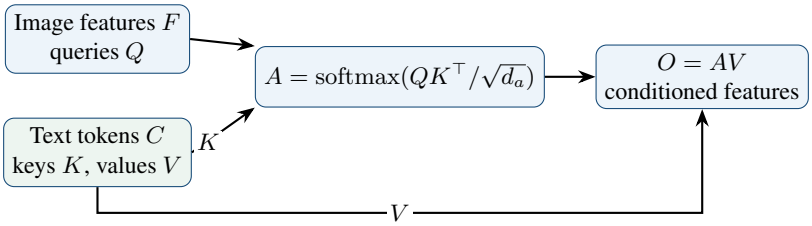


Figure 4. Original single-head cross-attention schematic, based on [10, 17]. Multi-head attention learns several such projections and combines their outputs.

D.1. Textbook VAE objective and the LDM distinction

A textbook negative evidence lower bound for a VAE is [8]

$$\begin{aligned}\mathcal{J}_{\mathrm{VAE}} = \mathbb{E}_{q_\phi(\tilde{z}|x_0)}[-\log p_\psi(x_0 \mid \tilde{z})] \\ + \mathrm{D}_{\mathrm{KL}}(q_\phi(\tilde{z} \mid x_0) \| \mathcal{N}(0, \mathbf{I}))\,.\end{aligned} \tag{28}$$

The decoder likelihood $p_\psi(x_0 \mid \tilde{z})$ rewards reconstruction; the nonnegative KL term regularizes the approximate posterior toward the prior. Natural logarithms give nats. This explains the VAE interpretation, but LDM's perceptual/adversarial first-stage objective with weak KL regularization is not identical to this pixel-likelihood objective [10, Sec. 3.1].

D.2. Cross-attention: how language reaches the denoiser

Use a *row-vector* convention. At one U-Net layer, flatten its feature map into $F \in \mathbb{R}^{n\times d_u}$, where n is the number of spatial positions and d_u is its channel width. Let $C = \tau_\eta(y) \in \mathbb{R}^{m\times d_y}$ contain m token embeddings of width d_y. One cross-attention head computes [10, 17]

$$Q = FW_Q, \quad K = CW_K, \quad V = CW_V, \tag{29}$$

$$A = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_a}}\right), \qquad O = AV. \tag{30}$$

The learned projections have shapes $W_Q \in \mathbb{R}^{d_u\times d_a}$, $W_K \in \mathbb{R}^{d_y\times d_a}$, and $W_V \in \mathbb{R}^{d_y\times d_v}$. Thus $Q \in \mathbb{R}^{n\times d_a}$ contains *queries from image features*, $K \in \mathbb{R}^{m\times d_a}$ contains *keys from text*, and $V \in \mathbb{R}^{m\times d_v}$ contains text values. d_a, d_v are positive integer head widths. A superscript $\top$ denotes transposition. Softmax acts across the m tokens for each spatial row, making $A \in \mathbb{R}^{n\times m}$ nonnegative with row sums of one. $O \in \mathbb{R}^{n\times d_v}$ supplies text-dependent features that are projected back into the U-Net's residual stream.

A numerical attention example

Take one spatial query $Q = (1, 0)$, two keys $K_1 = (\sqrt{2}, 0)$, $K_2 = (0, 0)$, and $d_a = d_v = 2$. The scaled logits are $(1, 0)$, so $A = (e/(e + 1), 1/(e + 1)) \approx (0.7311, 0.2689)$, where e is Euler's number. For $V_1 = (1, 0)$ and $V_2 = (0, 1)$, the output is $O \approx (0.7311, 0.2689)$. This original toy example shows weighted retrieval, not an observed attention map from an actual model.

D.3. Prompt control and classifier-free guidance

Cross-attention supplies the condition; *classifier-free guidance* (CFG) adjusts how strongly a conditional prediction influences sampling [5]. Train the same network with the condition replaced by a null condition $\varnothing$ with probability $p_{\mathrm{drop}} \in (0, 1)$. At inference define

$$\begin{aligned}\varepsilon_u &= \varepsilon_\theta(z_t, t, \tau_\eta(\varnothing)), \\ \varepsilon_c &= \varepsilon_\theta(z_t, t, \tau_\eta(y)), \\ \varepsilon_{\mathrm{cfg}} &= \varepsilon_u + s(\varepsilon_c - \varepsilon_u).\end{aligned} \tag{31}$$

$s \geq 0$ is a dimensionless guidance scale. Here $s = 0$ uses the unconditional prediction, $s = 1$ gives ordinary conditional sampling, and $s > 1$ extrapolates in the conditional direction. The CFG paper writes the equivalent expression $(1 + w)\varepsilon_c - w\varepsilon_u$; its parameter is $w = s - 1$. Stating the convention prevents confusing two scales with different numerical baselines.

For an original scalar illustration, $\varepsilon_u = 0.2$, $\varepsilon_c = 0.1$ and $s = 3$ give $\varepsilon_{\mathrm{cfg}} = -0.1$. Guidance is therefore an extrapolation, not a convex average. Substituting it into the reverse update changes the generation trajectory. Greater guidance can improve conditional fidelity while reducing diversity; it does not ensure monotonically better quality [5]. Standard CFG requires two denoiser evaluations per step, often batched together, adding compute and potentially peak memory.

For a proposed prompt-control experiment, including fixed seeds and separate attribute/relation changes, see Appendix B.